Meko feature proposal

Vaani: one memory per person, shared by every part of the assistant

Team Vaani: Aryan Thakur, Keshav Goel, Anish Manchanda · github.com/Aryannnthakurrr/Vaani · 1 November 2026

The product in one paragraph

Vaani is an Android voice companion for elderly, blind and non-technical people in India. They speak in Hindi, English or Hinglish, and Vaani uses their apps for them (Uber cabs, Pronto house help, shopping carts), finds and books a doctor by phoning hospitals, handles emergencies, warns about scams during calls, and checks food packs with the camera. Nothing irreversible happens until the person says “haan”. The AI suggests one step at a time; code on the phone checks every step.

Why context decides the outcome

Our users struggle most with repeating themselves and with multi-step choices. The same short sentence needs different actions for different people:

- **“Book a cab for my doctor’s appointment.”** The right destination is the clinic the person booked days earlier, through a different feature.
- **“Is this okay for me?” (holding a Frooti)** A diabetic should hear about the 7 teaspoons of sugar; someone without a diet condition should just hear the label.
- **“Aaj BP 146 by 92 aaya.”** The reply should compare it with this person’s doctor’s target and their own 7-day average, in their own language style.

Without memory the assistant must ask again, which is exactly what these users find hardest, or guess and get it wrong. With memory, one sentence is enough.

The design: one datapack per person, seven agents that record their source

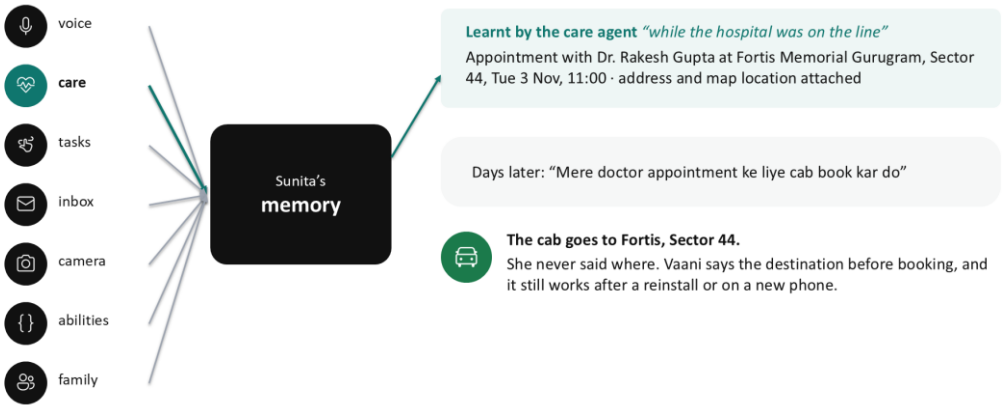


Figure: every part of Vaani writes into the person’s own Meko datapack as a named agent. A booking made by the doctor feature is later used by the cab feature.

Meko concept	How Vaani uses it	Why
Datapack	One per person (saathi-persona-sunita for demo users; a hashed id for real phones)	Search covers a whole datapack, so it is the privacy boundary. We measured this: two users sharing one datapack saw each other’s facts; with one each, neither did
Agent	Seven: saathi-voice, -care, -tasks, -inbox, -camera, -abilities, -family; each has its own long-lived profile conversation	One shared memory, but every fact is traceable to the part of the assistant that wrote it
Memory metadata	topic, key, agent, learned_from, plus structured fields (clinic, address, lat, lng, start time, booking reference)	A newer fact with the same key replaces the old one; other features can act on the structured fields
Conversation	One per compiled ability (a message per change) and one per phone task (a message per step)	Durable state and an audit trail, without storing screen or message text

What Meko does in Vaani

- **Cross-feature appointments.** After a confirmed booking, the care agent stores the appointment with its address and map location. “Book a cab for my doctor’s appointment” then sends the cab there and says the destination before booking. The phone asks the backend (GET /memory/appointment) if it has no local copy, so it still works after a reinstall or on a new phone.
- **Provenance users and families can see.** GET /memory/learned lists every fact with its agent and source, for example “learnt by the care agent while the hospital was on the line”, or “DHBVN bill SMS, summary only”. This makes the memory explainable and correctable.
- **State for personal compiled abilities.** Small tested programs keep each person’s bookkeeping: Ramesh’s sugar readings and tablet count, Sunita’s BP log and daily salt, Arjun’s expenses. Each ability’s state is mirrored to its own Meko conversation after every change, so the numbers stay with the person.
- **Personal context for the AI.** What Vaani knows about the person (language, conditions, doctor, next appointment) shapes the voice replies, the camera’s gentle sugar or salt nudge, and the doctor intake.
- **Phone-task logs without screen text.** Each task is a conversation of steps (action, reason, whether code or the AI decided). Screen text is never stored, only its size and the app.
- **Forgetting that works.** Keyed facts replace stale ones, and “forget everything” deletes the person’s facts in Meko too.

Privacy rules that stay in force

- **One datapack per person,** so a search never crosses people.
- **Never stored:** OTP messages and messages from saved contacts (they never leave the phone); the inbox agent stores summaries, never message text; the voice “remember” tool refuses OTPs, PINs, card or Aadhaar numbers.
- **Who sees what:** hospital calls get only the person’s name from memory; family alerts get no symptoms or amounts.

Built around Meko’s speed

Meko writes took 5–16 s and searches 15–18 s in our measurements (31 October 2026), so nobody waits on Meko. Writes land in a local store at once and reach Meko in the background; reads come from the local copy, and Meko is the durable, shared copy. Demo users are seeded on every backend start, adding only facts Meko does not have yet, and datapacks are found by name, so our three laptops share one datapack per person. Calls are limited to two at a time and retried after 5 s and 20 s when rate-limited.

Verified on the live service

Person	Datapack	Memories	Agents that wrote them
Ramesh, 74 (Hindi, diabetes)	saathi-persona-ramesh	11	care (2), tasks (2), inbox, camera, abilities, voice, family (3)
Sunita, 67 (Hinglish, BP)	saathi-persona-sunita	10	care (2), voice (2), inbox, camera, abilities, family (3)
Arjun, 41 (English, blind)	saathi-persona-arjun	10	care (2), tasks (2), voice (2), abilities, family (3)

Read back with memory_get_all; the appointment’s structured metadata (clinic, address, latitude and longitude, start time) came back intact. Behaviour is covered by backend tests (test_memory.py, test_voice_live.py, test_abilities.py).

Next steps

- **Read from Meko on the request path** so a brand-new backend recovers a person’s memory, not only writes and checks it.
- **Live writes from the camera and inbox agents** (today the voice, care and abilities agents write live; the demo users’ camera and inbox facts are seeded).
- **Meko artifacts for compiled skills** and token tracking, so skills and their cost travel with the person.